
JavaScript

Nous avons vu jusqu'à présent comment créer des pages Web en utilisant du HTML et du CSS. Mais comment faire pour créer des pages dynamiques ? C'est à dire des pages qui répondraient à des actions de l'utilisateur...

Côté serveur

Nous avons aperçu avec les formulaires qu'il est possible de créer une page sur un serveur qui récupère les données d'un formulaire pour utiliser des données passées en paramètre (soit avec la méthode GET soit avec la méthode POST). Le serveur est alors capable de "calculer" une page Web utilisant ces paramètres en utilisant le langage PHP (nous y reviendrons). L'inconvénient de cela est précisément de devoir recalculer et donc, de recharger toute la page... Comment faire si on souhaite juste changer la couleur d'un texte, une couleur de fond, ajouter ou supprimer une ligne de texte, par exemple ? En utilisant JavaScript, cela va pouvoir se faire sur le client : le navigateur qui visite le site, donc.

Côté client, JavaScript, premiers pas

En 1995, Brendan Eich, alors employé par la société Netscape Communication, crée un nouveau langage intégré à leur navigateur phare Netscape Navigator : Javascript. Ce navigateur n'a pas survécu à l'arrivée massive de Internet Explorer intégré au système d'exploitation Windows de Microsoft. Son code ouvert servira de base à Firefox qui est toujours aujourd'hui très bien placé dans les navigateurs les plus utilisés.

Dès 1996, JavaScript devient incontournable pour la programmation Web, tant sur Netscape Navigator que sur Internet Explorer qui a rapidement intégré sa propre version.

Pour se faire une première idée, entrez le code suivant :

```
<!DOCTYPE html>
<html>
  <head>
    <title>Clicks</title>
    <script type="text/javascript">
      var compteur = 0;
      function suivant() {
        compteur = compteur + 1;
        var v = document.getElementById("valeur");
        v.innerHTML = compteur;
      }
    </script>
  </head>
  <body>
    <div id="mondiv">
      <button onclick="suivant();">CLIC !</button>
      <span id = "valeur">0</span>
    </div>
  </body>
</html>
```

Enregistrez ce code HTML et ouvrez-le avec votre navigateur.

Ce code est utilisable directement dans un navigateur et ne nécessite aucune interaction avec le serveur.

Le corps du document ne comporte pas beaucoup d'éléments... Un simple bouton et un élément `<span>`, le tout dans une `<div>` (qui ne sera utile que pour la suite).

Ce qui est important, pour l'instant, c'est que l'élément `span` possède un ***id*** et que le bouton possède un attribut ***onclick***. La valeur de cet attribut est en fait un appel de fonction. Lorsqu'on clique sur le bouton, la fonction est appelée. C'est ce qu'on appelle de la programmation événementielle.

Si on regarde de plus près, dans la balise `<head>` la fonction ***suivant()*** est définie dans une balise `<script>`. Cette balise indique simplement au navigateur qu'on va lui donner du JavaScript !

Le fonctionnement de ce script est relativement simple :

- On définit une variable ***compteur***
- On définit ensuite la fonction ***suivant()***
- Quand cette fonction est appelée, on incrémente la variable
- On récupère l'élément sur lequel intervenir à partir de son ***id***
- on affiche la variable dans le code HTML de l'élément

En cliquant sur le bouton, le compteur s'incrémente, et le contenu du `<span>` est mis à jour avec la nouvelle valeur.

Pour avoir quelques autres exemples de ce que peut faire JavaScript, ouvrez la console JavaScript en appuyant sur F12 puis en cliquant sur l'onglet Console et tapez les codes suivants :

```
document.body.style.background = "red";
var d = document.getElementById("mondiv");
d.style.color = "yellow";
d.style.fontWeight = "bold";
d.style.border = "1px dashed white";
d.innerHTML = "Et si on <i>changeait tout</i> !...";
```

En rechargeant la page avec F5, ces modifications sont annulées et le compteur revient à 0.

Tout comme le CSS pour lequel il est recommandé d'utiliser un fichier séparé, le JavaScript peut être isolé dans un fichier spécifique. Du point de vue de la réutilisation du code, c'est très intéressant. Et du point de vue de la lecture du HTML, c'est également plus simple, ça compartimente mieux les choses... Ce sont de bonnes pratiques à respecter !

Ainsi, le code de votre premier script devra être dans un fichier que vous nommerez par exemple, `compteur.js` et dans la section `<head>` de votre fichier HTML, il suffit de faire un lien vers ce script. en modifiant le code de la façon suivante suivante.

Fichier compteur.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Clicks</title>
    <script type="text/javascript" src="compteur.js" defer="defer">
    </script>
  </head>
```

```
<body>
  <div id="mondiv">
    <button id="bouton">CLIC !</button>
    <span id = "valeur">0</span>
  </div>
</body>
</html>
```

Deux choses importantes sont à noter. L'attribut onclick a disparu et, en revanche, le bouton dispose d'un id. Ceci va impliquer des modifications sur le fichier JavaScript !

Le fichier **compteur.js** devient alors :

```
var compteur = 0;
function suivant() {
  compteur = compteur + 1;
  var v = document.getElementById("valeur");
  v.innerHTML = compteur;
}

var b = document.getElementById("bouton");
b.addEventListener("click", suivant);
```

Le début du script est identique mais à la fin, on ajoute un gestionnaire d'événements pour lui dire d'appeler la fonction suivant() lorsque l'utilisateur clique sur le bouton. Pour cela, on commence par sélectionner le bouton par son id et on appelle la fonction addEventListener pour ce bouton.

Il y a plusieurs avantages à procéder comme cela. De nombreux événements peuvent être traités, voire générés de façon dynamique. Il est possible de retirer un gestionnaire d'événements avec **removeEventListener()**. Et, non des moindres, plusieurs gestionnaires peuvent être définis pour le même événement.

```
var b = document.getElementById("bouton");
b.addEventListener("click", suivant);
b.addEventListener("click", fonction2);
```

Exercices

1 - Créez une page HTML contenant une zone de saisie du salaire brut avec l'ID **salaire**, un bouton de validation avec l'ID **valide** et quatre span également identifiés pour pouvoir y ajouter du texte.

Créez un fichier JavaScript qui va, en utilisant la valeur de

```
document.getElementById("salaire").value
```

qui contient la valeur saisie par l'utilisateur, afficher le salaire net et quelques charges quand on clique sur le bouton...

- Impôts sur le revenu : 18%
- Assurance maladie: 7%
- Régime de retraite : 5%

Vous devrez donc écrire un gestionnaire d'événements pour le bouton et la fonction **calculeSalaireNet()** qui reprendra la valeur saisie par l'utilisateur dans la zone **salaire**, fera les calculs et affichera les résultats dans les span.

Salaire brut :

Impots sur le revenu : 324

Cotisation assurance maladie : 126

Cotisation retraite : 90

Salaire Net : 1260

2 - En utilisant la fonction prédéfinie **Math.random()** qui renvoie un nombre entre 0 et 1, écrivez une fonction qui change la couleur de fond de la page de façon aléatoire à chaque clic sur le bouton.

3 - En utilisant notamment la fonction prédéfinie **setInterval(fonction, délai)** et l'objet **Date**, créez un script qui affiche l'heure qu'il est et qui la met à jour chaque seconde. C'est un peu plus compliqué, il va falloir vous aider de Google !

En cas de problème ou d'erreur, la console (**F12** et onglet **console**) utilisée tout à l'heure peut vous aider à debugger votre script... Ne l'oubliez pas !

Manipuler les classes

Nous avons vu comment accéder à un élément par son id en utilisant **getElementById**. On peut aussi accéder à une liste d'éléments en utilisant leur classe. Je vous rappelle que si un id identifie un élément unique, la classe peut en identifier plusieurs. La fonction **getElementsByClassName** va donc renvoyer plusieurs éléments dans un tableau.

Vous avez vu en Python, la structure syntaxique **for**. En Javascript, même si elle ne s'écrit pas de la même façon, elle existe aussi. Nous allons en avoir besoin pour parcourir le tableau contenant la liste des éléments retournés par la fonction. Pour voir comment cela fonctionne, créez les codes suivants, en ajoutant ce qui manque !

- **HTML : changeClass.html**

```
<!DOCTYPE html>
<html>
  <head>
    <!-- ajoutez ici tout le code nécessaire au JavaScript... -->
    <!-- ajoutez aussi tout le code nécessaire au CSS... -->
  </head>
  <body>
    <button id="bouton">Rouge !</button>
    <p class="vert">
      Ce texte est écrit en vert :
    </p>
    <p class="vert">
      Et celui ci aussi !
    </p>
  </body>
</html>
```

- **CSS : changeClass.css**

```
.vert {
  color: green;
}
```

- **JS : changeClass.js**

```
function ecritRouge() {
  var elements = document.getElementsByClassName('vert');

  for (var i = 0; i < elements.length; i++) {
    var element = elements[i]; // i-ème élément du tableau
    element.style.color = "red"
  }
}

//ajoutez ici le gestionnaire d'événements pour le bouton
```

Normalement, quand vous cliquez sur le bouton, tous les éléments verts deviennent rouges. On peut ainsi manipuler les éléments d'une classe donnée. Il est également simple d'ajouter ou de retirer des classes en utilisant les fonctions **add** et **remove**

```
element.classList.add("my-class");
element.classList.remove("my-class");
```

Ainsi on peut très bien imaginer le code suivant : Le fichier HTML reste identique, en revanche :

- CSS : changeClass.css

```
.vert {
  color: green;
}
.rouge {
  color: red;
}
```

- JS : changeClass.js

```
function ecritRouge() {
  var elements = document.getElementsByClassName('vert');

  for (var i = 0; i < elements.length; i++) {
    var element = elements[i]; // i-ème élément du tableau
    element.classList.remove("vert");
    element.classList.add("rouge");
  }
}

//ajoutez ici le gestionnaire d'événements pour le bouton
```

Suivant le navigateur, il peut se produire un résultat étrange... Les éléments changent de couleurs les uns après les autres... En fait, quand on supprime la classe **vert** de notre paragraphe, cela modifie la liste des éléments, il n'en reste plus qu'un et la boucle est terminée.

Pour que cela fonctionne, il vaut mieux sélectionner les éléments **p** et modifier leurs classes...

Cela peut se faire facilement. Nous avons vu comment sélectionner les éléments par leur **id** ou par leur **class**. Il est également possible d'utiliser la fonction **getElementsByTagName()** afin de sélectionner tous les éléments d'une balise donnée. Ici, nous voulons sélectionner toutes les balises **paragraphe** (donc **p**).

Au lieu de :

```
var elements = document.getElementsByClassName('vert');
```

Il suffira d'écrire

```
var elements = document.getElementsByTagName('p');
```

afin de sélectionner tous les paragraphes.

Maintenant, le code fonctionne comme attendu ! Les classes **vert** sont supprimées, les classes **rouge** sont ajoutées. Quel que soit le navigateur !

Retour sur la boucle for

Dans le programme précédent, vous avez utilisé le code JavaScript suivant :

```
for (var i = 0; i < elements.length; i++) {  
    ...  
}
```

On va indiquer trois éléments séparés par des points-virgules :

- la valeur d'initialisation : `i = 0`
- la condition de fin : `i < elements.length`
- l'opération pour passer à la valeur suivante : `i++`

On va donc commencer par initialiser `i` à sa première valeur, donc 0, et on va faire les opérations comprises entre les accolades. Quand c'est fini, on passe à la valeur suivante de `i`.

Notez au passage l'opérateur d'incrémentation `++`, c'est la même chose que de faire `i = i + 1`, sauf que ça permet de taper moins de caractères !

Quand on a la valeur suivante de `i`, on teste la condition de fin. Si c'est ok (si `i` est toujours inférieur à la taille de la liste `elements`), on continue, sinon, on sort et on passe à la suite (ce qu'il y a après l'accolade fermante)

Pour mieux comprendre, entrez le code suivant...

Code HTML : boucleFor.html

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Boucle for</title>  
    <script type="text/javascript" src="boucleFor.js"></script>  
  </head>  
  
  <body>  
    Allez voir la console !!!  
  </body>  
</html>
```

Code JavaScript : boucleFor.js

```
console.log("Avant la boucle");  
for (i = 0; i < 10; i++) {  
    console.log(i);  
}  
console.log("Après la boucle");
```

Affichez votre page. Le body est presque vide, donc ne s'affichera que l'indication d'aller voir dans la console (une bonne habitude à prendre !).

`console.log` permet d'afficher des valeurs dans la console (ça peut toujours être utile !)

Vous devez y voir quelque chose comme ça :



Avant la boucle
0
1
2
3
4
5
6
7
8
9
Après la boucle

Dès que la valeur de `i` arrive à 10, elle n'est plus < 10 , donc on sort de la boucle.

Structure conditionnelle : `if`

De la même manière qu'en Python, il est très simple de mettre en place des conditions pour rediriger l'exécution vers un bloc ou un autre.

Par exemple, en Python, on écrira :

```
if age >= 18:
    print("Vous êtes majeur");
else:
    print("Vous êtes mineur");
```

En JavaScript, c'est presque pareil, sauf que le bloc, comme à chaque fois, sera encadré par des accolades au lieu d'être indenté. Il faudra également encadrer la condition avec des parenthèses. On écrira donc :

```
if (age >= 18) {
    console.log("Vous êtes majeur");
} else {
    console.log("Vous êtes mineur");
}
```

Exercice

1 – Écrivez le code permettant d'afficher des listes différentes en fonction du bouton sur lequel on clique.

Pour vous aider, voici le code HTML complet et le début du JS, il reste à terminer !

Code HTML

```
<!DOCTYPE html>
<html>
  <head>
```



```

<title>Exercice boucle for</title>
<script type="text/javascript" src="exFor.js" defer></script>
</head>

<body>
  <button id="animaux">Animaux</button>
  <button id="fleurs">Fleurs</button>
  <ul id="liste">
  </ul>
</body>
</html>

```

Code JS

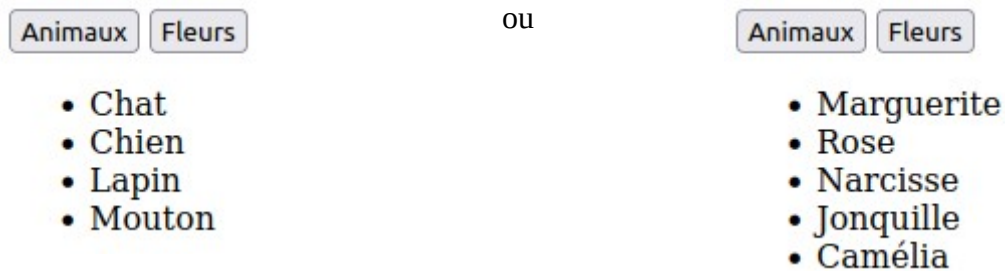
```

var animaux = ["Chat", "Chien", "Lapin", "Mouton"];
var fleurs = ["Marguerite", "Rose", "Narcisse", "Jonquille", "Camélia"];
var liste = document.getElementById("liste");

...

```

Votre page HTML devra ressembler à ça :

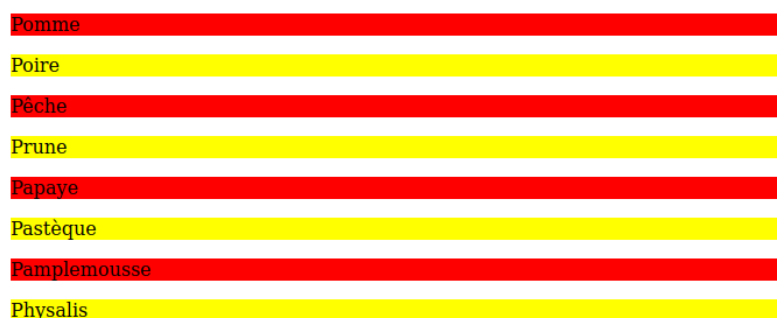


En cliquant sur le bouton **Animaux**

En cliquant sur le bouton **Fleurs**

Un indice : Pensez à la propriété innerHTML qui peut être écrite **ET** lue...

2 – Ecrivez un code JS permettant de réaliser la page suivante



Pour cela, voici le code HTML

```

<!DOCTYPE html>
<html>
  <head>
    <title>Exercice 2 boucle for</title>
    <script type="text/javascript" src="exFor2.js" defer></script>
  </head>

```

```

<body>
  <div>
    <p class="alterne">Pomme</p>
    <p class="alterne">Poire</p>
    <p class="alterne">Pêche</p>
    <p class="alterne">Prune</p>
    <p class="alterne">Papaye</p>
    <p class="alterne">Pastèque</p>
    <p class="alterne">Pamplemousse</p>
    <p class="alterne">Physalis</p>
  </div>
</body>
</html>

```

A vous de faire le fichier JS !

Indice : Comme en python, il existe l'opérateur % pour déterminer si une valeur est paire ou impaire. En effet, c'est le reste de la division entière. Si $i \% 2 == 0$, le nombre est pair, sinon, il est impair.

Une autre boucle : while

L'instruction **while** permet de créer une boucle qui s'exécute tant qu'une condition de test est vérifiée. La condition est évaluée avant d'exécuter l'instruction contenue dans la boucle. Il est donc possible de ne jamais passer dans la boucle...

```

var n = 0;

while (n < 3) {
  n++;
}

console.log(n);

```

Normalement, dans la console, vous aurez la valeur 3... En effet n est initialisé à 0 et **tant que (while)** n est strictement inférieur à 3, on l'augmente. Quand il arrive à 3, on sort de la boucle et on l'affiche. C'est très proche d'une boucle for, tout ça ! Ca permet quand même de faire beaucoup d'autres choses...

prompt

La méthode **window.prompt()** affiche une boîte de dialogue, éventuellement avec un message, qui invite l'utilisateur à saisir un texte.

```

resultat = window.prompt("Quel est ton signe astrologique", "Verseau");

```

Va ainsi afficher une boîte de dialogue demandant à l'utilisateur de saisir son signe astrologique en proposant comme valeur par défaut "Verseau" ! Quand l'utilisateur valide, le texte saisi se retrouve dans la variable resultat.

Par exemple, le code HTML suivant :

```

<!DOCTYPE html>
<html>
  <head>

```

```
<title>Exemple de prompt</title>
<script type="text/javascript" src="exPrompt.js" defer></script>
</head>

<body>
  Affiche le résultat de prompt dans la console...
</body>
</html>
```

et le javascript associé :

```
resultat = window.prompt("Quel est ton signe astrologique", "Verseau");
console.log(resultat)
```

Essayez aussi sans mettre le **defer**, pour bien voir la différence !

Exercice 1 : while et prompt

Ecrivez le code HTML et JS permettant de n'afficher une page que si l'âge saisi est supérieur ou égal à 18.

Tant que la valeur saisie est inférieure à 18, le système redemande l'age.

Exercice 2 : while et prompt

Écrivez le code HTML et JS permettant d'afficher les membres de la suite de Syracuse dans la console.

Rappel : Pour une valeur n , **si n est paire la suivante est $n/2$ sinon, la suivante est $3*n+1$** . On continue les calculs jusqu'à arriver à la valeur 1. Par exemple :

3, 10, 5, 16, 8, 4, 2, 1 est la suite commençant à 3.

Vous devrez la valeur de départ avec prompt puis afficher **dans la console** toutes les valeurs de la suite jusqu'à 1.

Une dernière boucle : do ... while

Nous avons vu la boucle while, et plus particulièrement qu'il est possible de ne jamais passer par la boucle.

```
var n = 10;

while (n < 3) {
  n++;
}

console.log(n);
```

Au moment de faire le test `while`, n est déjà supérieur à 3, il ne passera donc pas par le `n++` et affichera directement la valeur de n , soit 10.

Ceci impose une contrainte dans certains cas. Dans l'exercice 1 précédent, où il fallait vérifier l'âge de la personne avant d'autoriser ou non l'accès, cela impose de commencer par demander l'âge avant de faire le

premier test de la boucle while. Il faut donc écrire deux fois le prompt... Cela peut être bien si on souhaite changer le message mais pas forcément. Ce serait bien d'avoir une boucle dans laquelle le test serait fait à la fin ! Eh bien c'est l'objet de cette nouvelle boucle : `do ... while` !

Avec le `while`, nous avons :

```
age = window.prompt("Quel est votre âge ?");  
while (age < 18) {  
    age = prompt("Quel est votre âge ?");  
}
```

Avec le `do ... while`, nous aurons :

```
do {  
    age = prompt("Quel est votre âge ?");  
} while (age < 18)
```

L'age est testé uniquement après l'exécution du code. Le contenu de la boucle est donc exécuté au moins une fois. Il n'est alors plus nécessaire de commencer par demander l'âge, ça sera fait dans la boucle.

On peut ainsi forcer l'exécution d'un code. Reprenons l'exemple du début...

```
var n = 10;  
  
do {  
    n++;  
} while (n < 3)  
  
console.log(n);
```

Cela affichera cette fois 11 et plus 10 dans la console, parce que le `n++` est exécuté quoiqu'il arrive.

Exercice 1

Écrire un script qui donne le plus grand diviseur commun (PGDC) de deux nombres entiers différents de 0 entrés par l'utilisateur. Utilisez pour cela l'algorithme d'Euclide, algorithme qui s'énonce ainsi sur Wikipedia "On considère que **$pgcd(a,0) = a$** et que **si $b \neq 0$ $pgcd(a,b) = pgcd(b, a \% b)$** . On progresse dans l'algorithme en diminuant à chaque étape les nombres considérés par calcul du modulo".

En d'autres mots : "On commence donc par calculer le reste de la division de a par b , qu'on note r ; puis on remplace a par b , puis b par r , et on réapplique le procédé depuis le début. On obtient ainsi une suite, qui vaut 0 à un certain rang ; le PGCD cherché est le terme précédent de la suite."

Il faudra donc deux prompts pour les valeurs de a et b (le plus grand entré d'abord) et une boucle `do...while` pour faire les calculs jusqu'à arriver à 0

Exercice 2

Créez un script qui affiche dans la console la moyenne d'une série de notes entrées au clavier par l'utilisateur. La saisie s'arrête quand la note saisie est -1. Cette fois, il faudra une boucle `do...while`, un prompt et un `if`.

Attention, prompt renvoie une chaîne de caractères...